

Session 5 — Maîtriser les licences libres

Objectifs de la séance

- Lire et interpréter une licence courante (MIT, GPL, Apache).
- Évaluer la compatibilité entre licences.
- Choisir une licence adaptée à un projet.
- Utiliser les standards (SPDX) et outils.
- Comprendre DCO, CLA et les mécanismes de contribution.

Partie 1 — Les licences permissives

Philosophie

▮ « *Faites ce que vous voulez avec ce code, tant que vous me créditez.* »

Caractéristiques : très peu de restrictions, aucune obligation de partager les modifications, compatibles avec du code propriétaire, perçues comme *business-friendly*.

Principales : **MIT**, **BSD** (2 ou 3 clauses), **Apache 2.0**, **ISC**.

MIT — la plus simple et la plus populaire

Texte de 170 mots. Droits accordés très larges : utiliser, copier, modifier, fusionner, publier, distribuer, sous-licencier, vendre. Obligations minimales : **conserver le copyright et la licence** dans les copies. Limitations : aucune garantie, aucune responsabilité.

Projets emblématiques : jQuery, **React**, Rails, Node.js, .NET Core.

BSD — 2, 3 ou 4 clauses

Version	Contenu	Compatibilité
BSD 4-cl (années 1980, 4.3BSD)	1. clause « pas de pub avec mon nom » + « mentionner Berkeley dans la pub »	Incompatible GPL
BSD 3-cl (<i>New BSD</i> , 1999)	Berkeley retire la clause de publicité le 22 juillet 1999	Compatible GPL
BSD 2-cl (<i>Simplified</i>)	Encore plus simple, ≈ MIT	Compatible

Projets : FreeBSD, OpenBSD, nginx.

Apache 2.0

Plus complète que MIT/BSD (2 000 mots), avec des protections supplémentaires :

- **Grant explicite de brevets** : quiconque distribue du code Apache 2.0 accorde automatiquement une licence sur les brevets qu'il détient sur ce code.
- **Protection contre les litiges brevets** : toute action en contrefaçon brevetaire contre le projet entraîne perte de la licence pour l'attaquant.
- Définitions claires, fichier **NOTICE** requis si présent, documentation des modifications.

Projets : **Android (AOSP)**, **Kubernetes**, TensorFlow, Swift.

Comparatif

Critère	MIT	BSD 3-cl	Apache 2.0
Longueur	170 mots	220	2 000
Attribution	✓	✓	✓

Critère	MIT	BSD 3-cl	Apache 2.0
Protection brevets	✗	✗	✓
Fichier NOTICE	✗	✗	✓
Complexité	Très faible	Faible	Moyenne

Partie 2 — Les licences copyleft

Philosophie

« Vous pouvez utiliser ce code librement, mais vos modifications doivent rester libres. »

Caractéristiques : libertés **virales**, œuvres dérivées soumises à la même licence, empêche la « privatisation » du code, plus restrictif pour l'usage commercial mixte.

Œuvre dérivée vs œuvre collective

Question centrale pour comprendre le copyleft :

Œuvre dérivée	Œuvre collective (agrégation)
Modification, adaptation, transformation	Assemblage d'œuvres indépendantes
L'œuvre originale est <i>incorporée</i>	Chaque partie reste distincte
Le copyleft s'applique à l'ensemble	Le copyleft ne « contamine » pas les autres
Ex : fork, modification du code	Ex : distribution Linux (kernel + apps)

Copyleft fort vs copyleft faible

Copyleft fort (GPL, AGPL)	Copyleft faible (LGPL, MPL)
Tout programme dérivé doit être libre	Seules les modifications de la lib sont concernées
Linker avec du GPL → tout devient GPL	Possible de linker avec du code propriétaire
Perçu comme « contaminant »	Compromis pour bibliothèques

GPL (GNU General Public License)

Créée par Stallman. Trois versions :

- **v1** (1989) — première version.
- **v2** (1991) — la plus utilisée historiquement (**Linux kernel**).
- **v3** (2007) — ajoute grant explicite de brevets, anti-DRM (**anti-Tivoization**), compatibilité Apache 2.0.

Conditions principales : inclure le **code source** avec toute redistribution, conserver la licence et les copyrights, **pas de restrictions supplémentaires**. Si vous distribuez un logiciel basé sur du GPL, **tout doit être redistribué sous GPL**.

Tivoization : TiVo utilisait du Linux (GPL v2) mais verrouillait matériellement ses appareils, empêchant l'utilisateur d'installer ses propres versions modifiées. GPL v3 interdit cette pratique.

AGPL (Affero GPL)

Problème visé : la GPL ne s'applique que lors de la **distribution**. Un service SaaS n'est pas distribué → pas d'obligation de partager le code, même modifié.

AGPL comble la faille : si vous offrez le logiciel **comme service réseau**, vous devez fournir le code source aux utilisateurs, même sans distribution de binaires.

Projets : Mastodon, Nextcloud, Grafana, Bluemind, OnlyOffice (cf. actualité Euro Office, session 8).

LGPL (Lesser GPL)

Copyleft **faible** conçu pour les bibliothèques :

- La bibliothèque elle-même et ses modifications **restent libres**.
- Le programme qui l'utilise peut être **propriétaire**, à condition de linker **dynamiquement** (le plus souvent).

Projets : glibc, Qt (partiellement), GTK.

Le débat du *linking*

Un programme qui utilise une bibliothèque GPL est-il une œuvre dérivée ? Question non tranchée en jurisprudence, position FSF :

Type de linking	Position FSF	Position alternative
Static linking	Œuvre dérivée	Consensus
Dynamic linking	Œuvre dérivée	Débattu (œuvre collective ?)
Import (Python, JS...)	Œuvre dérivée	Très débattu
Appel réseau / API	Non dérivée	Consensus

En pratique : suivre l'interprétation de l'auteur, ou utiliser LGPL/MPL si on veut éviter la question.

MPL (Mozilla Public License)

Copyleft **au niveau du fichier** :

- Les **fichiers modifiés** restent sous MPL.
- Les **nouveaux fichiers** peuvent être propriétaires.
- Compromis clair, compatible Apache 2.0.

Projets : Firefox, Thunderbird.

Partie 3 — Compatibilité des licences

Règle générale

Quand on combine du code sous plusieurs licences, **les obligations se cumulent**. Certaines combinaisons sont impossibles (obligations contradictoires), d'autres forcent la licence finale du résultat.

Permissif → **Copyleft** : **OK**. On peut intégrer du code permissif dans un projet copyleft — ex. MIT → GPL , BSD → GPL , Apache 2.0 → GPL v3 .

Copyleft → **Permissif** : **NON**. On ne peut pas réintégrer du GPL dans du MIT ou du propriétaire.

Le copyleft « contamine » : dès qu'on intègre du code GPL, le résultat doit rester GPL.

Matrice simplifiée

Source ↓ / Dest →	MIT	Apache 2.0	GPL v2	GPL v3	LGPL	AGPL
MIT						
Apache 2.0						
GPL v2						
GPL v3						

Source ↓ / Dest →	MIT	Apache 2.0	GPL v2	GPL v3	LGPL	AGPL
LGPL	✗	✗	✓	✓	✓	✓
AGPL	✗	✗	✗	✗	✗	✓

* GPL v2 *only* est incompatible avec GPL v3 ; Apache 2.0 est compatible avec LGPL v3 mais pas LGPL v2.1. Une licence GPL « or later » est beaucoup plus souple.

Conseils pratiques

1. **Vérifier la licence** avant d'intégrer du code externe (outil ou lecture directe).
2. **Documenter** les licences de toutes les dépendances.
3. **Éviter** de mélanger GPL v2 *only* avec d'autres licences.
4. **Préférer** les licences avec clause *or later*.
5. En cas de doute : consulter un expert ou un outil (FOSSA, FOSSology...).

Partie 4 — Choisir une licence

Règle d'or : ne créez pas votre propre licence

Pourquoi : risques juridiques (texte non testé), incompatibilité, confusion pour les utilisateurs, travail de rédaction complexe. Recommandation de l'OSI : « *Il y a déjà trop de licences. N'en ajoutez pas.* »

Questions à se poser

1. **Objectif** : que veut-on accomplir ?
2. **Communauté** : qui va contribuer ? Des entreprises ?
3. **Écosystème** : quelles licences utilisent les projets similaires ?
4. **Compatibilité** : avec quoi le code va-t-il être combiné ?
5. **Modèle économique** : monétisation prévue ?

Guide simplifié

Situation	Choix typique
Adoption maximale, y compris en propriétaire	MIT ou Apache 2.0
Garantir que le code reste libre	GPL
Bibliothèque, usage propriétaire accepté, améliorations libres	LGPL
Service web, anti-exploitation SaaS sans contribution	AGPL
Compromis au niveau du fichier	MPL 2.0

Ressource pratique : <https://choosealicense.com> (utile mais simpliste).

Partie 5 — Standards et outils

SPDX (Software Package Data Exchange)

Standard pour identifier et décrire les licences, normalisé **ISO/IEC 5962:2021**.

Identifiants courants : MIT, Apache-2.0, GPL-3.0-only, GPL-3.0-or-later, LGPL-2.1-only, BSD-3-Clause, MPL-2.0.

Usage : dans `pyproject.toml`, `package.json`, `Cargo.toml`, `pom.xml`, et en **header de fichiers source**.

Expressions SPDX

Pour les cas multi-licences :

GPL-2.0-only OR MIT # Au choix
Apache-2.0 AND MIT # Les deux s'appliquent
GPL-2.0-or-later WITH Classpath-exception-2.0 # Avec exception

Header recommandé

SPDX-License-Identifier: GPL-3.0-or-later
SPDX-FileCopyrightText: 2026 Your Name <you@example.com>

Outils

Outil	Type	Usage
REUSE (FSFE)	Open source	Vérification des métadonnées licence/copyright
licensee	Open source	Détection automatique (utilisé par GitHub)
ScanCode	Open source	Analyse complète
FOSSology	Self-hosted	Audit entreprise
FOSSA	SaaS	Compliance CI/CD
Snyk	SaaS	Sécurité + licences

Partie 6 — Contribuer à un projet

Qui détient le copyright sur les contributions ?

Par défaut : le contributeur. Conséquences pour le projet :

- Des **dizaines ou centaines de détenteurs de droits** sur une base de code.
- **Impossibilité de changer de licence** sans l'accord de tous.
- Difficulté à poursuivre les violations.
- Identification floue des contributeurs.

Trois mécanismes pour clarifier la situation : **DCO, CLA, Copyright Assignment.**

DCO — Developer Certificate of Origin

Déclaration **légère** : en signant son commit (`git commit -s`), le contributeur certifie qu'il a le droit de soumettre ce code sous la licence du projet. Ajoute une ligne `Signed-off-by:` dans le message de commit.

Utilisé par : **Linux kernel**, CNCF, GitLab, Docker. C'est le mécanisme le plus simple et le moins intrusif.

CLA — Contributor License Agreement

Le contributeur **conserve son copyright** mais accorde une **licence large** au projet (souvent large et irrévocable).

Avantages : licence perpétuelle et irrévocable ; permet le *relicensing* ultérieur ; facilite les poursuites en cas de violation ; sensibilise aux questions de PI.

Inconvénients : friction pour les nouveaux contributeurs ; souvent perçu négativement ; paperasse administrative.

Utilisé par : **Apache**, Google, Microsoft.

Copyright Assignment

Le contributeur **transfère la propriété** de son copyright au projet. Le contributeur n'est plus titulaire ; reçoit parfois un *license back*.

Utilisé historiquement par la **FSF** pour les projets GNU. Rare ailleurs (trop intrusif).

Comparatif

Mécanisme	Propriété conservée	Relicensing
Rien	Oui	Impossible sans accord de tous
DCO	Oui	Impossible sans accord de tous
CLA	Oui (licence accordée)	Possible*
Assignment	Non (transfert)	Possible

* Si le CLA le permet explicitement.

GitHub et *inbound* = *outbound*

Les conditions d'utilisation de GitHub précisent : « *Whenever you add Content to a repository containing notice of a license, you license that Content under the same terms.* » En pratique, contribuer via PR sur GitHub = accepter la licence du projet.

Partie 7 — Cas spéciaux

Dual licensing

Un même projet disponible sous **deux licences au choix** (généralement GPL + licence commerciale). Classique : **MySQL**, **Qt**. Nécessite le contrôle du copyright (CLA). Source de revenus (cf. session 9).

Exceptions de licence

Permissions spéciales ajoutées à une licence existante :

- **Classpath Exception** (Java, utilisée par OpenJDK) : autorise à linker du code non-GPL avec les bibliothèques de la plate-forme sans que le tout soit soumis à la GPL.
- **GCC Runtime Exception** : permet de distribuer un exécutable compilé par GCC sans que la **libgcc** (liée à l'exécutable) ne déclenche le copyleft sur le reste du programme.
- **Font Exception** : les documents utilisant une police libre ne sont pas GPL.

Domaine public

Domaine public « naturel » : copyright expiré (70 ans post-mortem). Rare pour le logiciel (trop récent).

Dédication volontaire : **CC0** (Creative Commons Zero), **Unlicense**, **WTFPL**.

Subtilité : en **France** et **Allemagne**, on ne peut pas vraiment renoncer au copyright. D'où l'utilité de CC0 qui accorde une licence très permissive **en fallback**.

Creative Commons — pour le contenu, pas le code

Variante	Signification	Libre ?
CC0	Domaine public	Oui
CC BY	Attribution	Oui
CC BY-SA	Attribution + <i>ShareAlike</i> (copyleft)	Oui
CC BY-NC	Non-commercial	Non
CC BY-ND	No Derivatives	Non

Usage : documentation, assets, datasets. **Pas recommandé pour le code.**

Licences source available (pas open source)

SSPL (MongoDB), BSL (HashiCorp, MariaDB), **Elastic License, Commons Clause**. Motivation : protection contre les hyperscalers (« AWS-proofing »), monétisation directe. **Ces licences ne sont pas open source selon l'OSI** — sujet central de la session 9.

Relicensing (changement de licence)

Possible si vous détenez 100 % des droits (auteur unique, salariés d'une même organisation, cession par les contributeurs, CLA qui le permet) OU si la destination est **compatible** (ex : MIT → GPL).

Impossible si des contributeurs multiples n'ont pas donné leur accord explicite, ou si du code tiers sous copyleft est incorporé.

Cas célèbres : MongoDB (AGPL → SSPL), Elastic (Apache 2.0 → SSPL/ELv2), HashiCorp (MPL → BSL), modules Redis. Réaction communautaire systématique : **forks** (OpenSearch, OpenTofu, Valkey...).

Partie 8 — Étude de cas : Chardet (2026)

Les faits

chardet : bibliothèque Python de détection d'encodage, 130 M téléchargements/mois. Mainteneur : Dan Blanchard depuis 12 ans (auteur original : Mark Pilgrim). En 2026, **version 7.0** : **réimplémentée par IA (Claude)**, 48× plus rapide, multi-cœurs. Licence changée de **LGPL** → **MIT**. Méthode : l'API et les tests sont fournis à l'IA, sans lire le code source — similarité JPlag < 1,3 %.

La controverse

Position du mainteneur (Dan Blanchard) : il s'agit d'une réimplémentation *clean room* via IA, le code est entièrement nouveau, il n'y a donc pas d'obligation de conserver la LGPL. L'amélioration (performance) justifie le relicensing.

Objection de l'auteur original (Mark Pilgrim) : le mainteneur a été exposé de façon massive au projet existant ; il y avait un contrat social implicite avec les contributeurs ; la LGPL protégeait leurs apports ; le *relicensing* constitue une rupture de confiance.

Question centrale : une réimplémentation par IA est-elle une « œuvre nouvelle » ou une « œuvre dérivée » ?

Trois points de vue

- **Armin Ronacher** (créateur de Flask) — favorable : « *la GPL va contre l'esprit du partage.* » Voit l'IA comme libératrice.
- **Antirez** (créateur de Redis) — favorable : « *GNU a réimplémenté UNIX, Linux a réimplémenté via Minix. L'IA accélère un processus qui a toujours existé.* »
- **Zoë Kooyman** (FSF) — opposée : « *Refuser aux autres les droits que vous avez reçus est profondément antisocial, quelle que soit la méthode.* »

Les enjeux

Arguments pro-réimplémentation : c'est légalement permis ; il y a un précédent historique (GNU a réimplémenté Unix, Linux a réimplémenté Minix) ; l'IA accélère l'innovation ; cela rééquilibre le rapport de force face aux géants de l'industrie.

Arguments anti-réimplémentation : légal ≠ légitime ; la direction du changement (copyleft → permissif) inverse l'esprit du libre ; le contrat social entre contributeurs est brisé ; la protection offerte par le copyleft s'érode.

Ironie notée par plusieurs observateurs : **Vercel a réimplémenté Bash (GPL) en MIT** pour un de ses produits, mais s'est offusqué quand Cloudflare a fait pareil avec Next.js (MIT).

Questions ouvertes

1. Le droit d'auteur protège-t-il les **spécifications** (API, tests, comportement) ?
2. Le code généré par IA est-il **protégeable** ? (pas de jurisprudence claire en 2026)
3. Faut-il un **copyleft de spécification** (TGPL proposé par certains) ?
4. La **vitesse** (5 jours au lieu de 5 ans) change-t-elle la nature du processus ?

Références :

- Discussion GitHub : <https://github.com/chardet/chardet/issues/327>
- *The Ship of Theseus* — Armin Ronacher : <https://lucumr.pocoo.org/2026/3/5/theseus/>
- *AI and OSS licensing* — Antirez : <https://antirez.com/news/162>
- *Legal vs Legitimate* — Hong Minhee : <https://writings.hongminhee.org/2026/03/legal-vs-legitimate/>

Ce qu'il faut retenir

1. **Permissif** (MIT, Apache) → peu de contraintes, adoption maximale.
2. **Copyleft** (GPL) → garantit que le code reste libre.
3. **Copyleft faible** (LGPL, MPL) → compromis pour bibliothèques.
4. **Compatibilité** : permissif → copyleft OK, inverse **NON**.
5. **SPDX** : standard pour identifier les licences.
6. **DCO / CLA** : mécanismes de contribution.
7. **Ne créez jamais votre propre licence.**

Tableau récapitulatif

Licence	Type	Copyleft	Brevets	Usage typique
MIT	Permissive	Non	Non	Libs, frameworks
Apache 2.0	Permissive	Non	Oui	Corporate, cloud
GPL v2	Copyleft fort	Oui	Implicite	Linux kernel
GPL v3	Copyleft fort	Oui	Oui	Outils GNU
AGPL	Copyleft + SaaS	Oui	Oui	Services web
LGPL	Copyleft faible	Partiel	Oui	Bibliothèques
MPL 2.0	Copyleft fichier	Partiel	Oui	Mozilla

Checklist avant de publier un projet

- Fichier LICENSE à la racine.
- Identifier SPDX dans `pyproject.toml/package.json/Cargo.toml`.
- Headers SPDX dans les fichiers source.
- CONTRIBUTING.md expliquant DCO/CLA si applicable.
- Liste des dépendances et licences vérifiée.
- Compatibilité des licences validée.

Pour aller plus loin

- Open Source Definition : <https://opensource.org/osd>
- SPDX : <https://spdx.dev>
- REUSE (FSFE) : <https://reuse.software>